SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool: Scenario-Based Task (Medium)

Assessment Tool Weightage: 20%

QUESTIONS

QUESTIONS		
Q.No	**Question**	**Bloom's Level**
1	A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF	. BL4 – Analyze
2	A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach	. BL6 – Create
3	Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.	BL5 – Evaluate
4	A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.	BL6 – Create

5	A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.	BL4 – Analyze
6	Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.	BL3 – Apply
7	A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.	BL4 – Analyze
8	A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints.	BL5 – Evaluate
9	For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy.	BL6 – Create
10	A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.	BL5 – Evaluate

Code of Conduct:

I, Judy Allen J (192520055) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Judy Allen J

1. A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF.

Answer:

A university stores student, course, faculty, and marks information in a single table as shown below.

StudentID	StudentName	Department	CourseID	CourseName	FacultyID	FacultyName	Marks
S101	Alice	CSE	C201	DBMS	**F01**	Dr. Kumar	85
S101	Alice	CSE	C202	Operating Systems	F02	Dr. Priya	90
S102	Bob	IT	C201	DBMS	F01	Dr. Kumar	78

Analysis of the Existing Structure

The above table combines student, course, faculty, and enrollment details into a single relation. This design causes several normalization problems.

Issues Identified:

- Student details are repeated for every course enrollment, causing data redundancy.
- Faculty information is repeated for every student taking the same course.
- Updating a student's or faculty's information requires multiple row updates (Update Anomaly).
- A new course or faculty cannot be inserted without enrolling a student (Insertion Anomaly).
- Deleting the last student enrolled in a course removes course and faculty information (Deletion Anomaly).
- StudentName and Department depend only on StudentID, while CourseName depends only on CourseID (Partial Dependency).
- FacultyName depends on FacultyID instead of the primary key (Transitive Dependency).

First Normal Form (1NF)

The table is converted into 1NF by ensuring that all attributes contain atomic values and there are no repeating groups.

Enrollment Table

StudentID	CourseID	Marks
S101	C201	85
S101	C202	90
S102	C201	78

Second Normal Form (2NF)

To remove partial dependencies, the table is decomposed into separate relations.

Student Table

StudentID	StudentName	Department
S101	Alice	CSE
S102	Bob	IT

Course Table

CourseID	CourseName	FacultyID
C201	DBMS	F01
C202	Operating Systems	F02

Enrollment Table

StudentID	CourseID	Marks
S101	C201	85
S101	C202	90
S102	C201	78

Third Normal Form (3NF)

To remove transitive dependencies, faculty details are stored in a separate table.

Student Table

StudentID	StudentName	Department
S101	Alice	CSE
S102	Bob	IT

Faculty Table

FacultyID	FacultyName
F01	Dr. Kumar
F02	Dr. Priya

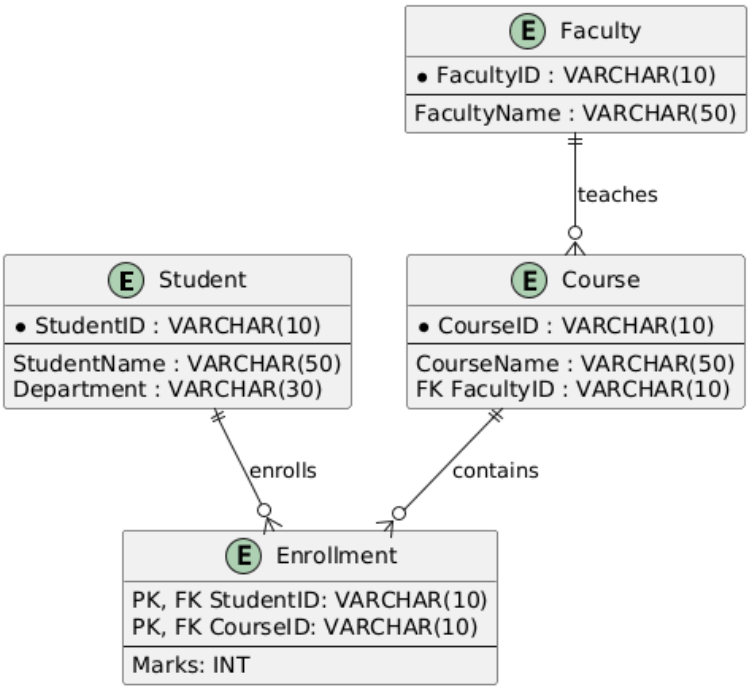
Course Table

CourseID	CourseName	FacultyID
C201	DBMS	F01
C202	Operating Systems	F02

Enrollment Table

StudentID	CourseID	Marks
S101	C201	85
S101	C202	90
S102	C201	78

Final 3NF Schema



Conclusion

The redesigned database satisfies Third Normal Form (3NF) because all partial and transitive dependencies have been eliminated. Each table stores information about a single entity, reducing redundancy and preventing update, insertion, and deletion anomalies. This design improves data integrity, consistency, and maintenance, making the database more efficient and scalable.

2. A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach.

Answer:

A company maintains employee and department information in the following tables.

Employee Table

EmployeeID	EmployeeName
E101	John
E102	Alice
E103	Bob

Department Table

DepartmentID	DepartmentName
D01	HR
D02	Finance
D03	IT

EmployeeDepartment Table

EmployeeID	DepartmentID
E101	D01
E101	D02
E102	D03
E103	D01
E103	D03

The EmployeeDepartment table stores the relationship between employees and departments. An employee can work in one or more departments.

Analysis of the Scenario

The objective is to identify employees who are assigned to more than one department.

Key Issues Identified

- An employee may belong to multiple departments.
- The relationship between employees and departments is many-to-many.
- The EmployeeDepartment table is used to maintain these relationships.

- SQL aggregation is required to count the number of departments assigned to each employee.

SQL Query

```
mysql> SELECT
    ->     e.EmployeeID,
    ->     e.EmployeeName,
    ->     COUNT(ed.DepartmentID) AS TotalDepartments
    -> FROM Employee e
    -> JOIN EmployeeDepartment ed
    -> ON e.EmployeeID = ed.EmployeeID
    -> GROUP BY e.EmployeeID, e.EmployeeName
    -> HAVING COUNT(ed.DepartmentID) > 1;
```

Explanation of the Query

- JOIN combines the Employee and EmployeeDepartment tables using EmployeeID.
- COUNT(DepartmentID) calculates the number of departments assigned to each employee.
- GROUP BY groups records based on EmployeeID and EmployeeName.
- HAVING COUNT(DepartmentID) > 1 filters only those employees who work in more than one department.

Sample Output

```
+------------+--------------+------------------+
| EmployeeID | EmployeeName | TotalDepartments |
+------------+--------------+------------------+
| E101       | John         |                2 |
| E103       | Bob          |                2 |
+------------+--------------+------------------+
2 rows in set (0.00 sec)
```

The output shows that John and Bob are working in multiple departments.

Justification of the Approach

The SQL query uses JOIN, GROUP BY, and HAVING clauses to accurately retrieve employees assigned to multiple departments. The JOIN operation connects employee records with their department assignments, GROUP BY organizes records employee-wise, and COUNT() determines the number of departments each employee belongs to. The HAVING clause filters only employees whose department count is greater than one. This approach is efficient, avoids redundant processing, and produces accurate results even for large databases.

Conclusion

The designed SQL query successfully retrieves employees working in multiple departments by using SQL aggregation and filtering techniques. It provides an efficient solution for analyzing employee-department assignments and supports better workforce management. This approach is scalable, accurate, and follows standard SQL practices.

3. Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.

Answer:

Consider the following relation:

R(StudentID, StudentName, CourseID, CourseName, FacultyID, FacultyName)

Functional Dependencies (FDs)

1. StudentID → StudentName
2. CourseID → CourseName, FacultyID
3. FacultyID → FacultyName
4. StudentID, CourseID → All Attributes

Analysis of the Relation

The relation stores information about students, courses, and faculty in a single table. The given functional dependencies show that some attributes depend on other non-key attributes, which may lead to redundancy and anomalies.

Functional Dependency Analysis

- StudentID determines StudentName.
- CourseID determines CourseName and FacultyID.
- FacultyID determines FacultyName.
- The combination of StudentID and CourseID uniquely identifies every record.

Determining the Candidate Key

To identify every tuple uniquely, both StudentID and CourseID are required.

Candidate Key

(StudentID, CourseID)

Since neither StudentID nor CourseID alone can determine all attributes, the composite key (StudentID, CourseID) is the only candidate key.

Checking BCNF

A relation is in Boyce-Codd Normal Form (BCNF) if, for every non-trivial functional dependency X → Y, X is a super key.

Checking Each Functional Dependency

Functional Dependency	Is Left Side a Super Key?	BCNF Status
StudentID → StudentName	No	Violates BCNF
CourseID → CourseName, FacultyID	No	Violates BCNF
FacultyID → FacultyName	No	Violates BCNF
StudentID, CourseID → All Attributes	Yes	Satisfies BCNF

Since three functional dependencies have determinants that are not super keys, the relation does not satisfy BCNF.

BCNF Decomposition

The relation is decomposed into smaller relations as follows.

Student Table

StudentID	StudentName
S101	Alice
S102	Bob

Faculty Table

FacultyID	FacultyName
F01	Dr. Kumar
F02	Dr. Priya

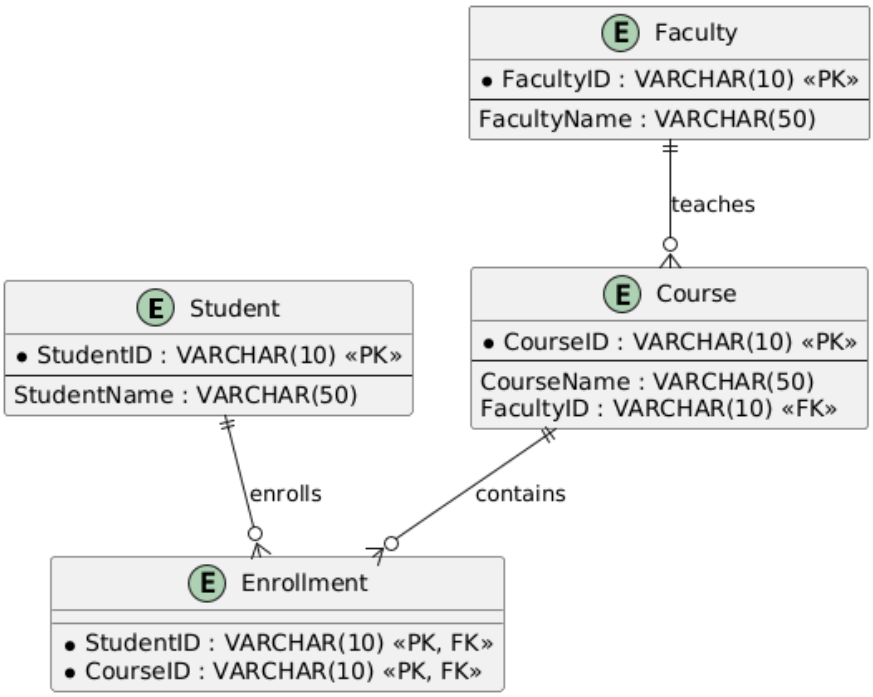
Course Table

CourseID	CourseName	FacultyID
C201	DBMS	F01
C202	Operating Systems	F02

Enrollment Table

StudentID	CourseID
S101	C201
S101	C202
S102	C201

Final BCNF Schema



Justification

The decomposition removes all BCNF violations by ensuring that every determinant is a candidate key in its respective relation. Each table represents a single entity, eliminating redundancy and preventing update, insertion, and deletion anomalies. The decomposition is lossless and improves data consistency, integrity, and maintainability.

Conclusion

The candidate key of the original relation is (StudentID, CourseID). The original relation does not satisfy BCNF because several functional dependencies have determinants that are not super keys. After decomposition into Student, Faculty, Course, and Enrollment tables, all relations satisfy BCNF, resulting in an efficient and well-structured database design.

4. A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.

Answer:

A banking system stores customer, account, and transaction information. Different users such as customers, bank tellers, and managers require different levels of access to the database. To ensure data security and maintain data integrity, SQL Views and Constraints are used.

Analysis of the Scenario

The banking database contains sensitive information such as customer details, account balances, and transaction records. Unauthorized users should not have access to confidential data. Database constraints help maintain valid data, while views restrict access based on user roles.

Objectives

- Protect sensitive customer information.
- Restrict access according to user roles.
- Maintain data accuracy and consistency.
- Prevent invalid or unauthorized data entry.

Sample Tables

Customer Table

CustomerID	CustomerName	AccountNo	Balance	Branch
C101	John	A1001	50000	Chennai
C102	Alice	A1002	75000	Bangalore

SQL View for Customers

This view allows customers to view only their account details without exposing confidential internal information.

```
mysql> CREATE VIEW CustomerView AS
    -> SELECT CustomerID,
    ->        CustomerName,
    ->        AccountNo,
    ->        Balance
    -> FROM Customer;
Query OK, 0 rows affected (0.05 sec)
```

```
mysql> SELECT * FROM CustomerView;
+------------+--------------+-----------+----------+
| CustomerID | CustomerName | AccountNo | Balance  |
+------------+--------------+-----------+----------+
| C101       | John         | A1001     | 50000.00 |
| C102       | Alice        | A1002     | 75000.00 |
| C103       | Bob          | A1003     | 35000.00 |
| C104       | David        | A1004     | 92000.00 |
+------------+--------------+-----------+----------+
4 rows in set (0.00 sec)
```

SQL View for Bank Managers

This view provides managers with complete customer information.

```
mysql> CREATE VIEW ManagerView AS
    -> SELECT *
    -> FROM Customer;
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> SELECT * FROM ManagerView;
+------------+--------------+-----------+----------+----------+
| CustomerID | CustomerName | AccountNo | Balance  | BranchID |
+------------+--------------+-----------+----------+----------+
| C101       | John         | A1001     | 50000.00 | B01      |
| C102       | Alice        | A1002     | 75000.00 | B02      |
| C103       | Bob          | A1003     | 35000.00 | B03      |
| C104       | David        | A1004     | 92000.00 | B01      |
+------------+--------------+-----------+----------+----------+
4 rows in set (0.00 sec)
```

SQL Constraints

- **Primary Key Constraint**

  Ensures every customer has a unique CustomerID.

  CustomerID INT PRIMARY KEY

- **NOT NULL Constraint**

  Ensures mandatory fields are not left empty.

  CustomerName VARCHAR(50) NOT NULL

- **UNIQUE Constraint**

  Ensures that every account number is unique.

  AccountNo VARCHAR(20) UNIQUE

- **CHECK Constraint**

  Prevents negative account balances.

  Balance DECIMAL(10,2)

CHECK (Balance >= 0)

- **FOREIGN KEY Constraint**

  Maintains referential integrity between Customer and Branch tables.

  FOREIGN KEY (BranchID)

  REFERENCES Branch(BranchID)

```
mysql> CREATE TABLE Branch (
    ->     BranchID VARCHAR(10) PRIMARY KEY,
    ->     BranchName VARCHAR(50),
    ->     Location VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.09 sec)

mysql> CREATE TABLE Customer (
    ->     CustomerID VARCHAR(10) PRIMARY KEY,
    ->     CustomerName VARCHAR(50) NOT NULL,
    ->     AccountNo VARCHAR(20) UNIQUE,
    ->     Balance DECIMAL(10,2) CHECK (Balance >= 0),
    ->     BranchID VARCHAR(10),
    ->     FOREIGN KEY (BranchID) REFERENCES Branch(BranchID)
    -> );
```

Justification of the Approach

The use of Views restricts users from accessing unauthorized information by displaying only the required columns for each user role. Customers can view only their account details, while managers have access to complete records. Constraints such as PRIMARY KEY, NOT NULL, UNIQUE, CHECK, and FOREIGN KEY ensure data integrity by preventing duplicate entries, null values, invalid balances, and inconsistent relationships between tables. Together, views and constraints provide a secure, reliable, and well-structured banking database.

Conclusion

The designed views and constraints effectively enhance data security and data integrity in the banking system. Views implement role-based access control, while constraints enforce valid and consistent data. This design ensures secure access, minimizes errors, and improves the overall reliability of the banking database.

5. A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.

Answer:

A sales database stores customer, product, sales, and salesperson information in a single table as shown below.

Sales Table (Unnormalized Relation)

SaleID	CustomerID	CustomerName	ProductID	ProductName	SalespersonID	SalespersonName	Quantity
S001	C101	John	P101	Laptop	SP01	David	2
S002	C101	John	P102	Mouse	SP02	Alice	5
S003	C102	Bob	P101	Laptop	SP01	David	1

Analysis of the Existing Structure

The single table stores customer, product, salesperson, and sales information together, resulting in redundancy and data anomalies.

Issues Identified

- Customer details are repeated for every purchase.
- Product details are repeated whenever the product is sold.
- Salesperson information is repeated for every sale.
- Updating customer, product, or salesperson details requires modifying multiple records (Update Anomaly).
- A new customer or product cannot be inserted until a sale is made (Insertion Anomaly).
- Deleting the last sale of a product removes product information (Deletion Anomaly).
- CustomerName depends only on CustomerID.
- ProductName depends only on ProductID.
- SalespersonName depends only on SalespersonID.

First Normal Form (1NF)

The relation is converted into 1NF by ensuring all attributes contain atomic values.

Sales Table

SaleID	CustomerID	ProductID	SalespersonID	Quantity
S001	C101	P101	SP01	2
S002	C101	P102	SP02	5
S003	C102	P101	SP01	1

Second Normal Form (2NF)

To remove partial dependencies, separate tables are created.

Customer Table

CustomerID	CustomerName
C101	John
C102	Bob

Product Table

ProductID	ProductName
P101	Laptop
P102	Mouse

Salesperson Table

SalespersonID	SalespersonName
SP01	David
SP02	Alice

Sales Table

SaleID	CustomerID	ProductID	SalespersonID	Quantity
S001	C101	P101	SP01	2
S002	C101	P102	SP02	5
S003	C102	P101	SP01	1

Third Normal Form (3NF)

The decomposed tables already remove transitive dependencies. Each non-key attribute depends only on its table's primary key.

Customer Table

CustomerID	CustomerName
C101	John
C102	Bob

Product Table

ProductID	ProductName
P101	Laptop
P102	Mouse

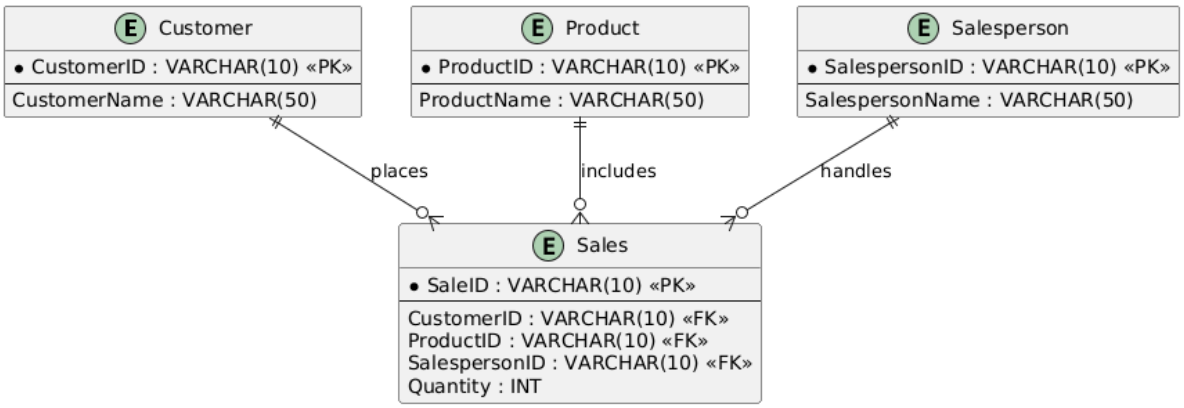
Salesperson Table

SalespersonID	SalespersonName
SP01	David
SP02	Alice

Sales Table

SaleID	CustomerID	ProductID	SalespersonID	Quantity
S001	C101	P101	SP01	2
S002	C101	P102	SP02	5
S003	C102	P101	SP01	1

Final 3NF Schema



Justification

The normalization process eliminates redundancy by separating customer, product, salesperson, and sales information into individual tables. This removes update, insertion, and deletion anomalies while improving data consistency and integrity. Every non-key attribute depends only on the primary key of its respective relation, satisfying the requirements of Third Normal Form (3NF).

Conclusion

The original sales database contained redundant information that caused data anomalies. After normalization, the database is decomposed into four well-structured relations: Customer, Product, Salesperson, and Sales. The redesigned schema satisfies Third Normal Form (3NF) by eliminating redundancy, improving maintainability, and ensuring efficient data management.

6. Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.

Answer

A company stores customer and order information in separate tables. The objective is to identify frequent customers, i.e., customers who have placed more than one order.

Customer Table

CustomerID	CustomerName	City
C101	John	Chennai
C102	Alice	Bangalore
C103	Bob	Hyderabad

Orders Table

OrderID	CustomerID	OrderDate	Amount
O001	C101	2026-08-01	5000
O002	C101	2026-08-02	3500
O003	C102	2026-08-03	4000
O004	C103	2026-08-04	2500
O005	C103	2026-08-05	4500

SQL Query Using JOIN

```
mysql> SELECT
    ->     c.CustomerID,
    ->     c.CustomerName,
    ->     COUNT(o.OrderID) AS TotalOrders
    -> FROM Customer c
    -> JOIN Orders o
    -> ON c.CustomerID = o.CustomerID
    -> GROUP BY c.CustomerID, c.CustomerName
    -> HAVING COUNT(o.OrderID) > 1;
```

Output

```
+------------+--------------+-------------+
| CustomerID | CustomerName | TotalOrders |
+------------+--------------+-------------+
| C101       | John         |           2 |
| C103       | Bob          |           2 |
+------------+--------------+-------------+
2 rows in set (0.00 sec)
```

SQL Query Using Subquery

```
mysql> SELECT CustomerID, CustomerName
    -> FROM Customer
    -> WHERE CustomerID IN
    -> (
    ->     SELECT CustomerID
    ->     FROM Orders
    ->     GROUP BY CustomerID
    ->     HAVING COUNT(OrderID) > 1
    -> );
+------------+--------------+
```

Output

```
+------------+--------------+
| CustomerID | CustomerName |
+------------+--------------+
| C101       | John         |
| C103       | Bob          |
+------------+--------------+
2 rows in set (0.00 sec)
```

Explanation

- The JOIN query combines the Customer and Orders tables using the CustomerID.
- The COUNT() function calculates the number of orders placed by each customer.
- The GROUP BY clause groups records based on each customer.
- The HAVING clause filters customers who have placed more than one order.
- The Subquery first identifies customer IDs with multiple orders and then retrieves their details from the Customer table.

Justification

The JOIN query efficiently combines customer and order information to generate a complete report of frequent customers. The Subquery provides an alternative approach by first identifying customers with multiple orders and then fetching their details. Both methods accurately identify frequent customers, improve query flexibility, and support business analysis for customer relationship management.

Conclusion

The SQL queries using JOIN and Subquery successfully retrieve frequent customers from the dataset. These queries help organizations identify valuable customers, analyze purchasing behavior, and support effective business decision-making.

7. A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.

Answer

Consider the following relation that stores student information, hobbies, and languages.

Student Table (Unnormalized Relation)

StudentID	StudentName	Hobby	Language
S101	Alice	Dancing	English
S101	Alice	Dancing	French
S101	Alice	Singing	English
S101	Alice	Singing	French
S102	Bob	Reading	English
S102	Bob	Reading	Hindi

Analysis of the Existing Structure

The relation stores student details, hobbies, and languages in a single table. A student may have multiple hobbies and may know multiple languages independently, creating multi-valued dependencies.

Multi-Valued Dependencies

- StudentID →→ Hobby
- StudentID →→ Language

Since hobbies and languages are independent of each other, combining them in one table creates unnecessary repetition.

Issues Identified

- Data redundancy due to repeated hobby and language combinations.
- Repeated student information in multiple rows.
- Update anomaly when modifying hobby or language details.
- Insertion anomaly when adding a new hobby or language.
- Deletion anomaly when removing the last hobby or language of a student.

Checking Fourth Normal Form (4NF)

A relation is in Fourth Normal Form (4NF) if it contains no non-trivial multi-valued dependencies other than those on a candidate key.

The given relation violates 4NF because:

- StudentID →→ Hobby
- StudentID →→ Language

Both are independent multi-valued dependencies.

Decomposition into 4NF

The relation is decomposed into two separate relations.

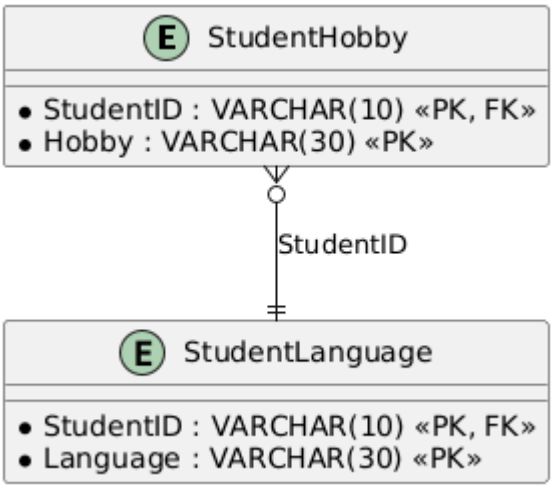
StudentHobby Table

StudentID	Hobby
S101	Dancing
S101	Singing
S102	Reading

StudentLanguage Table

StudentID	Language
S101	English
S101	French
S102	English
S102	Hindi

Final 4NF Schema



Justification

The decomposition separates hobbies and languages into independent relations, eliminating the multi-valued dependencies. Each table now represents a single independent relationship with the student. This removes redundancy, prevents update, insertion, and deletion anomalies, and ensures that every fact is stored only once. The resulting schema satisfies the requirements of Fourth Normal Form (4NF).

Conclusion

The original relation violated 4NF because independent multi-valued dependencies caused redundant data. By decomposing the relation into StudentHobby and StudentLanguage tables, all multi-valued dependencies are eliminated. The final database design satisfies Fourth Normal Form (4NF), improving data integrity, reducing redundancy, and making the database easier to maintain.

8. A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints.

Answer

Consider the following Employee table where improper constraints have resulted in inconsistent and invalid data.

Employee Table

EmployeeID	EmployeeName	Email	Salary	DepartmentID
E101	John	john@gmail.com	50000	D01
E102	Alice	alice@gmail.com	-2000	D02
E101	David	john@gmail.com	45000	D05

Evaluation of the Existing Schema

The database does not enforce appropriate integrity constraints, resulting in inconsistent and invalid data.

Problems Identified

- Duplicate EmployeeID values.
- Duplicate Email addresses.
- Negative salary values.
- Invalid DepartmentID that does not exist.
- Mandatory fields may contain NULL values.

These issues reduce data accuracy and compromise database integrity.

Suggested Integrity Constraints

1. PRIMARY KEY Constraint

Ensures every employee has a unique EmployeeID.

EmployeeID VARCHAR(10) PRIMARY KEY

2. NOT NULL Constraint

Ensures mandatory fields cannot be left empty.

EmployeeName VARCHAR(50) NOT NULL

3. UNIQUE Constraint

Ensures every employee has a unique email address.

Email VARCHAR(100) UNIQUE

4. CHECK Constraint

Ensures salary cannot be negative.

Salary DECIMAL(10,2)

CHECK (Salary >= 0)

5. FOREIGN KEY Constraint

Ensures employees are assigned only to valid departments.

FOREIGN KEY (DepartmentID)

REFERENCES Department(DepartmentID)

```
mysql> CREATE TABLE Department (
    ->     DepartmentID VARCHAR(10) PRIMARY KEY,
    ->     DepartmentName VARCHAR(50) NOT NULL
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Employee (
    ->     EmployeeID VARCHAR(10) PRIMARY KEY,
    ->     EmployeeName VARCHAR(50) NOT NULL,
    ->     Email VARCHAR(100) UNIQUE,
    ->     Salary DECIMAL(10,2) CHECK (Salary >= 0),
    ->     DepartmentID VARCHAR(10),
    ->     FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID)
    -> );
Query OK, 0 rows affected (0.09 sec)
```

Improved Table Design

Department Table

DepartmentID	DepartmentName
D01	HR
D02	Finance
D03	IT

Employee Table

EmployeeID	EmployeeName	Email	Salary	DepartmentID
E101	John	john@gmail.com	50000	D01
E102	Alice	alice@gmail.com	45000	D02
E103	David	david@gmail.com	60000	D03

Justification

Applying PRIMARY KEY, UNIQUE, NOT NULL, CHECK, and FOREIGN KEY constraints prevents duplicate records, invalid values, and inconsistent relationships between tables. These

constraints improve data accuracy, enforce business rules, and maintain referential integrity. As a result, the database becomes more reliable, secure, and easier to maintain.

Conclusion

The original schema suffered from inconsistent data because of missing integrity constraints. By implementing PRIMARY KEY, NOT NULL, UNIQUE, CHECK, and FOREIGN KEY constraints, the database ensures valid, consistent, and reliable data. These constraints improve overall database integrity and support efficient data management.

9. For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy.

Answer

Consider the following relation that stores supplier, product, and project information.

SupplierProductProject Table

SupplierID	ProductID	ProjectID
S1	P1	J1
S1	P2	J1
S2	P1	J2
S2	P2	J2

Analysis of the Existing Structure

The relation stores supplier, product, and project details in a single table. The relationship among these three attributes creates join dependencies, which may introduce redundancy and make the database difficult to maintain.

Join Dependency

The relation can be reconstructed by joining three smaller relations:

- Supplier supplies Products.
- Supplier works on Projects.
- Product is used in Projects.

This indicates the presence of a join dependency, meaning the table can be losslessly decomposed into smaller tables.

Issues Identified

- Redundant storage of supplier, product, and project combinations.
- Repeated data increases storage requirements.
- Update anomaly when modifying supplier-product or project information.
- Insertion anomaly when adding a new supplier or project.
- Deletion anomaly when removing existing records.

Checking Fifth Normal Form (5NF)

A relation is in Fifth Normal Form (5NF) if every join dependency is implied by the candidate keys and cannot be further decomposed without losing information.

The given relation violates 5NF because the join dependency is not based solely on the candidate key.

Decomposition into 5NF

The relation is decomposed into the following three relations.

SupplierProduct Table

SupplierID	ProductID
S1	P1
S1	P2
S2	P1
S2	P2

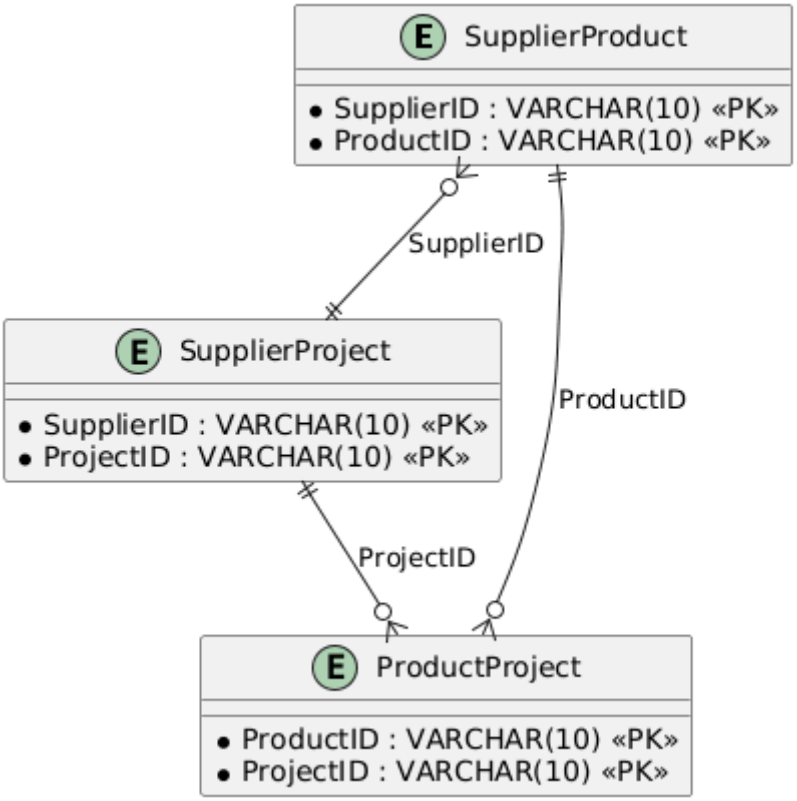
SupplierProject Table

SupplierID	ProjectID
S1	J1
S2	J2

ProductProject Table

ProductID	ProjectID
P1	J1
P2	J1
P1	J2
P2	J2

Final 5NF Schema



Justification

The decomposition separates the original relation into three independent relations based on the join dependency. Each table stores one relationship, eliminating redundant combinations of supplier, product, and project information. The original relation can be reconstructed using natural joins without any loss of information. Therefore, the decomposed schema satisfies Fifth Normal Form (5NF).

Conclusion

The original relation contained join dependencies that caused redundancy. By decomposing it into SupplierProduct, SupplierProject, and ProductProject tables, redundancy is eliminated while preserving all information. The resulting database satisfies Fifth Normal Form (5NF), improving consistency, reducing anomalies, and making the schema easier to maintain.

10. A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins,indexing concepts, or query restructuring.

Answer

A company maintains employee and department information in separate tables. The existing SQL query retrieves employee details using a subquery, which is inefficient for large datasets.

Employee Table

EmployeeID	EmployeeName	DepartmentID	Salary
E101	John	D01	50000
E102	Alice	D02	60000
E103	Bob	D01	55000

Department Table

DepartmentID	DepartmentName
D01	HR
D02	Finance
D03	IT

Analysis of the Existing Query

The following query retrieves employee and department details using a subquery.

Inefficient Query

```
mysql> SELECT EmployeeName,
    -> (
    ->     SELECT DepartmentName
    ->     FROM Department
    ->     WHERE Department.DepartmentID = Employee.DepartmentID
    -> ) AS DepartmentName
    -> FROM Employee;
+--------------+----------------+
```

Problems Identified

- The subquery executes once for every employee record.
- Increased execution time for large datasets.
- Higher CPU and memory usage.
- Poor query performance.

Optimized Query Using JOIN

```
mysql> SELECT
    ->     e.EmployeeID,
    ->     e.EmployeeName,
    ->     d.DepartmentName,
    ->     e.Salary
    -> FROM Employee e
    -> INNER JOIN Department d
    -> ON e.DepartmentID = d.DepartmentID;
+------------+--------------+----------------+----------+
| EmployeeID | EmployeeName | DepartmentName | Salary   |
+------------+--------------+----------------+----------+
| E101       | John         | HR             | 50000.00 |
| E102       | Alice        | Finance        | 60000.00 |
| E103       | Bob          | HR             | 55000.00 |
+------------+--------------+----------------+----------+
3 rows in set (0.01 sec)
```

Indexing Concept

```
mysql> CREATE INDEX idx_department
    -> ON Employee(DepartmentID);
Query OK, 0 rows affected (0.07 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

Query Restructuring

Instead of executing a subquery for every row, the JOIN operation retrieves matching records in a single operation, reducing execution time and improving efficiency.

Sample Output

```
+------------+--------------+----------------+----------+
| EmployeeID | EmployeeName | DepartmentName | Salary   |
+------------+--------------+----------------+----------+
| E101       | John         | HR             | 50000.00 |
| E102       | Alice        | Finance        | 60000.00 |
| E103       | Bob          | HR             | 55000.00 |
+------------+--------------+----------------+----------+
3 rows in set (0.01 sec)
```

Justification

The optimized query uses an INNER JOIN instead of a subquery, reducing the number of database operations. Creating an index on DepartmentID allows the database to locate matching records more quickly during join operations. Query restructuring improves readability, minimizes execution time, and enhances overall database performance, especially for large datasets.

Conclusion

The original query was inefficient because it executed a subquery for each row. By replacing it with an INNER JOIN, creating an index, and restructuring the query, data retrieval becomes faster and more efficient. These optimization techniques improve performance, reduce resource consumption, and support scalable database operations.